Projeto – Aplicações e Serviços na Web 2025/2026

Grupo 18

Joana Canana nº64158, José Lima nº64517, Pedro Ungari nº62261, Rodrigo Sequeira nº62255

1. Enquadramento

Apresentação do Projeto

A **WE DO** é uma aplicação para quem está cansado de combinar coisas no WhatsApp que nunca acontecem. O nosso objetivo é tirar os jovens do sofá e pô-los a fazer atividades reais (correr, jogar futebol, ir a um museu ou um café) através de um sistema de desafios e convites que exige uma "prova" no final. É uma plataforma onde o convite é um compromisso e a amizade é celebrada através de experiências reais, desde um café rápido até um jogo de futebol 11.

Problema a Resolver

- **A pessoa que diz que vai e depois não aparece:** É fácil dizer "eu vou" num chat e depois não aparecer.
- **Falta de Organização:** É difícil coordenar horários e locais com grupos grandes.
- **Isolamento:** Passamos horas a ver a vida dos outros no Instagram, mas poucas horas a viver a nossa com amigos.

Tipo de Aplicação

Rede Social Orientada a Ações (focada no "fazer" e não apenas no "publicar")

Reflexão Crítica (Fase 2):

O enquadramento, problema e motivação mantiveram-se consistentes ao longo desta fase. A visão teórica do projeto serviu como base sólida para definir a estrutura de dados e as rotas da API, garantindo que o backend em desenvolvimento responde diretamente à necessidade de combater o consumo passivo e promover ações reais.

2. Utilizadores e Cenários

Quem usa a WE DO?

- **Utilizador Comum (Membro/Líder/Apoiante):** Todos partilham a mesma base de permissões de "escrita" (criar ações, votar, validar).
- **Administrador:** O único com permissão de "moderação" (apagar conteúdos de terceiros e suspender contas).

Cenários de utilização principais

Cenário 1: Organizar uma Atividade de Grupo

- O Utilizador Comum que tem como objetivo criar e coordenar um evento desportivo informal.
- **Descrição:** O utilizador decide organizar um jogo de futebol. Através da WE DO, cria uma "Ação", define o local (via mapa), a hora e o número mínimo de participantes. Convida o seu "Squad" habitual. O sistema gere as confirmações em tempo real.

Cenário 2: Verificação por Pares e Prova de Ação

- O verificador (Amigo do participante) tem como objetivo validar a conclusão de uma tarefa para atribuição de pontuação.
- **Descrição:** Após um jantar de grupo ou uma caminhada, os participantes tiram uma fotografia coletiva no local. Um dos utilizadores submete a foto como "Prova de Ação". Os restantes participantes recebem uma notificação para **confirmar a presença** uns dos outros. A ação só é marcada como "Finalizada" e os pontos atribuídos quando o quórum de confirmação é atingido.

Cenário 3: Ativação Social via Sugestão Inteligente (uso da IA)

- O Utilizador Comum / Grupo. Tem como objetivo superar a indecisão logística usando inteligência artificial. O sistema utiliza um LLM para processar dados não estruturados (como o histórico de chat ou preferências do grupo) e gerar um objeto JSON que o sistema converte automaticamente num formulário de atividade
- **Descrição:** Um grupo de amigos está sem atividade planeada há vários dias. O sistema, detetando a inatividade e analisando o contexto (ex: tarde de sol, localização em Aveiro, preferência por atividades ao ar livre), sugere através do LLM uma "Missão": *"Caminhada no Passadiço com paragem na gelataria X"*. O utilizador pode converter esta sugestão numa "Ação" real com apenas um clique.

Cenário 4: Gestão e Governação de Comunidade

- O Administrador tem como objetivo moderar conteúdos e gerir a integridade da plataforma.
- **Descrição:** Um utilizador submete uma "Prova de Ação" que é denunciada por outros como sendo falsa ou imprópria. O Administrador acede ao painel de gestão, analisa a evidência e tem autoridade para anular a atividade, remover pontos ou suspender o utilizador.

Cenário 5: Sondagem para escolha da atividade

- Um grupo de amigos cria um grupo na app e estão indecisos no que fazer neste fim de semana, cada um sugere uma ação a fazer. As ações são recolhidas e cada um vota em

uma ou mais ações, a ação com mais votos será realizada e posteriormente pode ser partilhada na comunidade.

- **Descrição:** Um membro do "Squad" inicia uma sessão de planeamento para o fim de semana. Durante um período definido, cada amigo submete uma sugestão de "Ação" (ex: "Caminhada na Arrábida", "Torneio de Padel", "Visita ao Museu"). Cada sugestão inclui local e horário pretendido. O sistema consolida as propostas e abre a votação. Cada membro pode votar em uma ou mais opções que lhe agradem. Terminado o prazo, o sistema identifica a ação com mais votos e converte-a automaticamente num Evento Ativo do grupo, notificando todos os membros. Após a realização física da atividade, os participantes capturam uma fotografia de grupo. Os membros marcam-se mutuamente na fotografia. Uma vez que o grupo valide que a ação ocorreu conforme planeado, o registo é partilhado no feed da comunidade "WE DO" e os pontos de experiência são atribuídos aos perfis.

Reflexão Crítica (Fase 2):

Os cenários identificados na Fase 1 foram fundamentais para modelar o comportamento da base de dados nesta fase. Por exemplo, o Cenário de "Organizar uma Atividade de Grupo" guiou a criação das tabelas Squad e Action, enquanto o "Cenário de Verificação" motivou a implementação da tabela de junção Participation e da entidade Evidence no Prisma.

Storyboards

3. Funcionalidades

Funcionalidades Principais

- **Gestão de Identidade e Social:**
  - Registo e Autenticação de utilizadores (via JWT).
  - Criação e Gestão de "Squads" (grupos privados/públicos). Sistema de Perfis com métricas de atividade e "Streaks".

- **Motor de Atividades e Ações:**
  - Criação de Ações com integração de Mapas (localização/coordenadas).
  - Sistema de RSVPs (Confirmar/Rejeitar/Pendente).
  - Mecanismo de Sondagem: Proposta de múltiplas atividades e votação coletiva.

- **Validação e Prova social (Peer-Verification):**
    - Submissão de prova (upload de fotografia) via telemóvel.
    - Notificação de membros para validar a presença de pares.
    - Geofencing: Bloqueio de submissão de provas fora do raio de localização do evento (ainda não temos a certeza se vamos usar)
- **Inteligência e Descoberta (LLM):**
    - Sugestão de atividades baseada em meteorologia, localização e preferências.
    - Gerador de convites motivacionais automáticos.
- **Governação (Admin):**
    - Painel de moderação de conteúdos e gestão de utilizadores denunciados.

Reflexão Crítica (Fase 2):

As funcionalidades teóricas começaram agora a ser traduzidas em operações CRUD concretas na nossa API RESTful. A adição de propriedades geoespaciais (latitude e longitude) e campos temporais no nosso esquema de base de dados prepara o terreno para as futuras validações de geofencing e mecânicas de *streaks*.

Fluxos Principais do Sistema

Fluxo de Ciclo de Vida da Ação:

- **Criação:** O utilizador define os detalhes da Ação (onde, quando, quê).
- **Recrutamento:** O sistema dispara notificações para o Squad; os membros respondem com RSVP (Confirmado ou Recusado).
- **Execução e Prova:** Chegado o momento, um participante faz o upload da fotografia (A prova).
- **Consenso Social:** Os amigos marcados recebem um pedido de validação. Após a confirmação mútua, o estado da Ação muda para Finalizada.
- **Fecho:** O sistema calcula e atribui pontos de experiência e atualiza as *Streaks* (atividades consecutivas).

Fluxo de Decisão Coletiva (Sondagem):

- **Proposta:** Membros sugerem diferentes opções de atividades para o grupo.
- **Votação:** Os utilizadores votam nas suas preferências.
- **Resolução Automática:** O sistema encerra a votação e, usando o LLM (IA) em caso de empate ou indecisão, escolhe a melhor opção e converte-a automaticamente numa Ação agendada.

Recursos do Sistema

- Users (Utilizadores): Perfis, credenciais (Auth), nível de permissões (User/Admin) e estatísticas de gamificação.
- Squads (Grupos): Agrupamentos de utilizadores que partilham histórico e objetivos comuns.
- Actions (Ações/Eventos): O recurso central que guarda os metadados (título, categoria, coordenadas GPS, data/hora) e o estado atual.
- Participations (Participações): A tabela que liga utilizadores a ações, gerindo se o utilizador foi convidado, se confirmou e se a sua participação foi validada pelos pares.
- Evidences (Provas): Referências para as imagens armazenadas (Cloudinary) e os dados de validação social (quem aprovou a foto).
- Votes (Votos): Registos individuais de preferência dentro das sondagens de grupo.
- Badges/Achievements (Conquistas): Recursos estáticos que são atribuídos aos utilizadores quando atingem marcos (ex: "10 corridas em grupo").

4. Requisitos do Sistema

Requisitos Gerais

Categoria	Requisito	Descrição Técnica
Segurança	Autenticação JWT	Garantir que apenas utilizadores registados acedem às funcionalidades de escrita.
Dados	Persistência Relacional	Uso de PostgreSQL para gerir relações complexas entre Users, Squads e Actions.
Engenharia	Validação de Input	Implementação de esquemas de validação (ex: Zod) no backend para segurança dos dados.
Infraestrutura	Variáveis de Ambiente	Configuração de segredos e chaves de API via ficheiros .env.

Requisitos por Fase

- **Fase 1 (Atual):** Especificação completa, desenho da base de dados e planeamento da API.
- **Fase 2:** Implementação do Backend (Node.js), rotas de autenticação e CRUD base das Ações.
- **Fase 3:** Desenvolvimento do Frontend (React), integração de mapas e fluxo de submissão de provas (Evidence).
- **Fase 4:** Integração de funcionalidades LLM, sistema de gamificação, testes e documentação final (Swagger).

Reflexão Crítica (Fase 2):

Os requisitos de segurança e integridade de dados estipulados na fase anterior foram agora materializados através da integração do Prisma (para persistência e tipagem segura) e do Zod (para validação rigorosa de inputs), protegendo a nossa base de dados desde o momento em que o pedido chega à API.

5. Arquitetura Inicial

Exemplos Iniciais de Endpoints (API Preliminar):

Considerando as necessidades de gestão de identidade, ações, grupos e votações, a API deverá seguir uma arquitetura RESTful. Aqui estão alguns exemplos preliminares:

Utilizadores e Identidade:

- POST /api/users: Criação (registo) de um novo utilizador.
- POST /api/auth/login: Autenticação e geração do JSON Web Token (JWT) para gestão de estado *stateless* **Gestão de Ações (Eventos):**
- POST /api/actions: Criação de uma nova ação.
- GET /api/actions/:id: Obter detalhes de uma ação específica.
- PUT /api/actions/:id/participations: Atualização do estado RSVP (Confirmar/Rejeitar) de um utilizador.

Provas e Validação (Peer-Verification):

- POST /api/actions/:id/evidences: Submissão de uma fotografia como prova.
- PUT /api/evidences/:id/validations: Atualização do consenso social validando a prova submetida.

Sondagens (Decisão Coletiva):

- POST /api/squads/:id/polls: Criação de uma proposta de atividades para o grupo.
- POST /api/polls/:id/votes: Registo de um voto numa das opções.

Modelo de Dados Preliminar

O backend fará a ponte entre este modelo de domínio e a base de dados relacional:

- **Users:**
    - Guarda o perfil, credenciais, nível de permissões (User/Admin) e as métricas de gamificação (*streaks* e achievements);
    - Atributos: id, nome, email, password, role, pontos_experiencia, streak;
- **Squads:**
    - Agrupamentos de utilizadores que partilham atividades;

    - Atributos: id, nome_squad, data_criacao;

- **Actions:**
    - O recurso central que regista as atividades e o estado atual da atividade.
    - Atributos: id, titulo, categoria, descricao, num_participantes, latitude, longitude, data_hora, estado, squad_id;

- **Participations:**
    - Tabela de ligação entre Users e Actions, gerindo convites, respostas (RSVP) e validações sociais;
    - Atributos: id, action_id, rsvp_status, validacao;

- **Evidences:**
    - Guarda as referências para as imagens armazenadas externamente e os dados de quem as aprovou;
    - Atributos: id, action_id, user_id, imagem_url;

- **Polls:**
    - A sessão de planeamento iniciada por um membro de um grupo. Define o contexto geral da votação;
    - Atributos: id, squad_id, criador, data_inicio, data_limite, estado;

- **Votes:**
    - Registos individuais de preferência nas sondagens;
    - Atributos: id, poll_option, user_id, data_hora;

- **Achievements:**
    - Troféus e marcos de gamificação;
    - Atributos: id, nome, descricao;

Modelo de Dados Atualizado (Fase 2)

O nosso modelo de dados foi materializado utilizando o Prisma ORM sobre uma base de dados PostgreSQL. As principais entidades e relações incluem:

- **User:** Contém o registo, *roles*, *pontos_experiencia* e *streaks*. Relaciona-se com quase todas as entidades.
- **Squad:** Grupos criados por utilizadores, que agregam as *Actions* e *Polls*.
- **Action:** O núcleo do sistema, guardando título, categoria, descrição, coordenadas (latitude/longitude), estado e associação a um *Squad*.

- **Participation:** Tabela explícita que faz a ligação muitos-para-muitos entre *User* e *Action*, gerindo o RSVP e a validação social (*peer-verification*).
- **Evidence:** Armazena os URLs das fotografias submetidas como prova de conclusão da *Action*.
- **Poll e Vote:** Estruturas para a mecânica de decisão coletiva dentro de um *Squad*.
- **Achievement:** Sistema de conquistas (*badges*).

Endpoints Implementados (Fase 2):

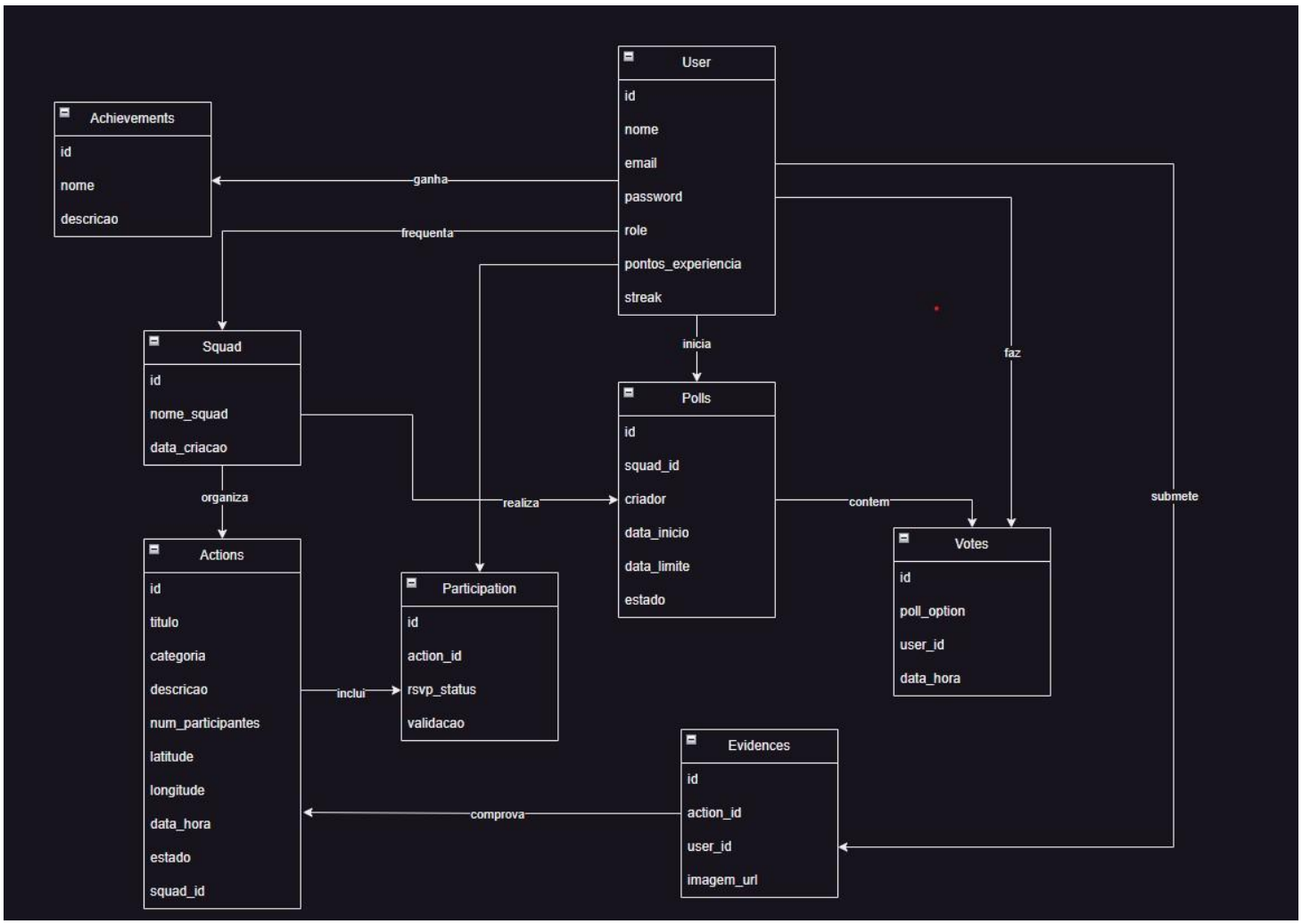
A API já se encontra funcional com os seguintes pontos de entrada (documentados no Swagger):

- GET /api/users: Listagem de todos os utilizadores registados.
- POST /api/users: Registo de novos utilizadores com validação rigorosa (Zod).
- GET /api/squads: Listagem de squads e ações associadas.
- POST /api/squads: Criação de novos grupos.
- POST /api/actions: Criação de uma nova atividade vinculada a um squad.

Reflexão Crítica (Fase 2):

O modelo preliminar da primeira fase foi expandido e formalizado num esquema Prisma maduro. Transformámos relações teóricas em tabelas físicas robustas (como a tabela explícita *Participation*) e adicionámos campos cruciais (data_hora, latitude, longitude) que faltavam no rascunho inicial, garantindo que o backend suporta todas as necessidades do *frontend*.

Diagrama Entidade Relacionamento



Serviços Externos Possíveis

- **Cloudinary:** Um serviço na cloud dedicado ao armazenamento, otimização e entrega de ficheiros multimédia. Vai alojar as fotografias submetidas pelos utilizadores no momento da prova que foi realizada a ação (Evidence). Na base de dados apenas são guardados os links ou URLs das imagens armazenadas no Cloudinary, evitando assim sobrecarregar a base de dados;
- **Google Maps API ou Mapbox:** Fornece mapas interativos, conversão de moradas em coordenadas (Geocoding) e cálculo de distâncias. É fundamental no momento da criação de Ações, permitindo ao utilizador definir o local exato da atividade de forma visual.
- **OpenWeatherMap**: Disponibiliza informações sobre o estado do tempo atual e previsões em tempo real para qualquer coordenada GPS.

Ideias iniciais para a funcionalidade com LLM

O cumprimento do requisito de integração de funcionalidades baseadas em LLM será aplicado no projeto "WE DO" para combater a inatividade dos grupos, reduzir impasses na realização de sondagens, assim como evitar fraudes durante o processo Evidence.

Ativação Social

- Conceito: O sistema utilizará o LLM para atuar como um organizador virtual do grupo. O modelo de IA processará dados não estruturados, combinando o histórico e as preferências do grupo com o contexto atual em tempo real (como a localização e a meteorologia).
- Execução técnica: Ao detetar vários dias de inatividade num "Squad", o LLM é acionado para criar uma "Missão". O LLM devolverá esta sugestão estruturada sob a forma de um objeto JSON, que o sistema converterá de forma transparente num formulário de atividade. O utilizador poderá então converter a sugestão numa Ação real apenas com um toque.

Resolução Automática de Sondagens

- Conceito: Visa otimizar o processo de Decisão Coletiva, abordando a situação em que as votações de grupo terminam sem um consenso claro.
- Execução técnica: Quando a votação é encerrada, se o sistema detetar um empate, é acionado o LLM. A Inteligência Artificial analisará as opções empatadas e o perfil do grupo para tomar a decisão final, escolhendo a melhor opção e convertendo-a automaticamente numa Ação, resolvendo o impasse logístico.

Pre-Validação de Provas

- Conceito: Utilizar um LLM para fazer uma triagem automática das fotografias submetidas, garantindo assim que ninguém ganha pontos com provas falsas.
- Execução técnica: Após o upload da fotografia para o Cloudinary, o backend envia o URL da imagem e o contexto da atividade ao LLM. O modelo avalia a imagem e devolve um JSON. Se for falso, o sistema bloqueia a submissão imediatamente, e dá um aviso ao Squad sobre o sucedido.

Principais Decisões Fase 2

Arquitetura em Camadas

Nesta fase, optou-se por implementar uma arquitetura de *software* estritamente dividida em camadas para garantir escalabilidade, manutenção e separação de responsabilidades. O código está organizado da seguinte forma:

- **Routes (/routes):** Responsáveis unicamente por definir os *endpoints* da API e reencaminhar os pedidos HTTP para o controlador adequado.

- **Controllers (/controllers):** Lidam com a lógica de HTTP (receber req, enviar res), processando a entrada do utilizador e comunicando com os serviços.
- **Services (/services):** Isolam a lógica de negócio e as interações com a base de dados (via Prisma). O controlador não sabe como os dados são guardados, apenas chama o serviço.
- **Middlewares (/middlewares):** Camada intermédia usada para intercetar pedidos (ex: validação de dados).

Justificação: Esta separação permite que diferentes elementos da equipa trabalhem em partes distintas do código sem causar conflitos. Além disso, se no futuro precisarmos de mudar de base de dados, apenas a camada de *Services* e o Prisma precisam de ser alterados, mantendo as *Routes* e os *Controllers* intactos.

Estratégias de Validação de Dados de Entrada

Para a validação dos dados (garantindo o requisito de segurança estipulado), implementámos a biblioteca **Zod**. Criámos ficheiros de *schema* (ex: user_schema.ts) que definem regras estritas para os dados esperados (ex: formato de e-mail, *strings* mínimas).

Para integrar isto elegantemente na API, desenvolvemos um *middleware* genérico (validate_middleware.ts) que corre antes do *Controller*.

Justificação: O uso do Zod aliado a um *middleware* previne que dados inválidos, incompletos ou maliciosos cheguem sequer aos nossos *Services* ou à base de dados. Se o pedido falhar na validação, o *middleware* aborta o processo automaticamente e devolve um erro 400 descritivo, poupando recursos do servidor e garantindo a integridade relacional no PostgreSQL.

Documentação Interativa

A documentação dos *endpoints* já finalizados foi implementada de forma interativa através do **Swagger UI** (disponível na rota /api-docs).

Justificação: Ter um contrato claro e visual da API é fundamental para as próximas fases. Facilita os testes de *backend* e servirá de guião direto quando começarmos a integrar o cliente Web (React) na Fase 3.

Estimativa de Conclusão

Requisito	Estado	Notas Atuais / O que falta
Requisitos Gerais		
Backend em Node.js (Express)	**100%**	Estrutura base implementada e a correr.
Persistência PostgreSQL (Prisma)	**100%**	Esquema Prisma finalizado e migrações criadas.

Modelo de dados coerente	**100%**	Todas as entidades da aplicação estão mapeadas.
Arquitetura em Camadas	**100%**	Separação entre Routes, Controllers e Services ativa.
Validação de *Inputs* (Zod)	**60%**	Middleware criado e ativo para *Users*. Falta estender aos restantes endpoints.
Documentação API (Swagger)	**70%**	Swagger configurado; documentados os endpoints de criação e listagem atuais. Falta documentar endpoints futuros.
Operações CRUD	**40%**	Criados POST e GET para *Users*, *Squads* e *Actions*. Falta desenvolver PUT/DELETE e CRUD para *Participations*, *Polls* e *Evidences*.
Autenticação (JWT)	**0%**	Planeado para o final da Fase 2 ou início da Fase 3.
Integração Frontend (React)	**0%**	Previsto para a Fase 3.
Integrações Externas / LLM	**0%**	Previsto para a Fase 4.
Requisitos Específicos (Projeto)		
Criação e Gestão de "Squads"	**70%**	Tabelas criadas; Endpoint básico funcional. Falta gestão avançada de membros.
Motor de Atividades (Actions)	**50%**	Modelo preparado com latitude/longitude; Endpoint de criação funcional. Falta associação de *evidences*.
Mecanismo de Sondagem (Polls)	**20%**	Modelo de dados criado no Prisma. Falta endpoints e lógica.